Tongji University

Design and Implementation of a Pipelined RISC-V x32 CPU

A report submitted for the course

Digital Electronics

Author:
Thierry Guichardaz

Professor:
Zhang, Zhifeng

Contents

1 Introduction **3**
1.1 Project Goal . . . 3
1.2 RISC-V ISA Overview . . . 3
1.3 Pipeline Architecture . . . 3

2 Design and Implementation **5**
2.1 Top-Level Architecture . . . 5
2.2 Pipeline Stages Implementation . . . 8
2.2.1 Instruction Fetch (IF) Stage . . . 8
2.2.2 Instruction Decode (ID) Stage . . . 9
2.2.3 Execute (EX) Stage . . . 10
2.2.4 Memory Access (MEM) Stage . . . 12
2.2.5 Write-Back (WB) Stage . . . 14
2.3 Key Functional Modules . . . 14
2.3.1 Control Logic (`maindec` and `aludec`) . . . 14
2.3.2 Data Memory (`dmem`) . . . 15
2.3.3 Arithmetic Logic Unit (`alu`) . . . 15
2.3.4 Hazard Unit (`hazard_unit`) . . . 16

3 Verification and Results **18**
3.1 Test Methodology . . . 18
3.2 Test Cases and Analysis . . . 19
3.2.1 Case 1: Data Forwarding from MEM to EX . . . 19
3.2.2 Case 2: Load-Use Hazard and Pipeline Stall . . . 20
3.2.3 Case 3: Control Hazard (Branch Taken) . . . 21
3.2.4 Case 4: Final Program Verification . . . 22

4 Conclusion **24**
4.1 Summary of Work . . . 24
4.2 Challenges and Future Work . . . 24

Abstract

This report details the design, implementation, and verification of a 32-bit, 5-stage pipelined processor based on the RISC-V instruction set architecture (ISA). The processor is implemented in SystemVerilog and supports a comprehensive subset of the RV32I base integer instruction set, including arithmetic, logical, memory access, and control flow instructions.

The design follows the classic 5-stage pipeline model: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB). To ensure correct execution, a complete hazard management system was implemented. Data hazards are resolved using a forwarding unit that bypasses data from the EX and MEM stages, complemented by a stall mechanism for load-use dependencies. Control hazards from branch and jump instructions are handled by flushing the initial stages of the pipeline.

The CPU's functionality was verified through a SystemVerilog testbench. The internal signals and register values were monitored and analyzed using GTKWave, confirming that the processor correctly executes sequences of instructions and that the hazard logic operates as intended.

1 Introduction

The field of digital electronics culminates in the design of complex systems like central processing units (CPUs), the core of modern computing. This report documents the process of creating such a system from the ground up, providing a practical link between theoretical concepts and a tangible, functional hardware implementation.

1.1 Project Goal

The primary objective of this project is to design, implement, and verify a 32-bit pipelined CPU. The goal is to apply the principles of computer architecture and digital logic learned during the course to a real-world design challenge. Using SystemVerilog, a hardware description language (HDL), we translate the architectural specifications of a processor into a digital circuit. The project focuses on correctly implementing instruction execution, managing data flow through a pipeline, and handling the associated hazards to ensure program correctness.

1.2 RISC-V ISA Overview

The processor is designed based on the RISC-V Instruction Set Architecture (ISA). An ISA defines the interface between software and hardware, specifying the set of instructions a processor can execute. RISC-V was chosen for this project for several key reasons: it is an open-source standard, it is modern, and its base instruction set is designed to be simple and easy to learn. This makes it an ideal platform for academic purposes. This project implements a significant subset of the RV32I, the base 32-bit integer instruction set, which is sufficient to run meaningful programs.

1.3 Pipeline Architecture

To improve performance, modern processors execute instructions in a pipeline. Instead of processing one instruction to completion before starting the next, a pipelined processor overlaps the execution of multiple instructions. This project implements the classic 5-stage RISC pipeline, which allows for a theoretical throughput of one instruction per clock cycle (IPC) in ideal conditions. The five stages are:

1. **Instruction Fetch (IF):** Fetches the next instruction from memory using the address stored in the Program Counter (PC).
2. **Instruction Decode (ID):** Decodes the fetched instruction to determine the operation to be performed and reads the required values from the register file.
3. **Execute (EX):** Performs the main operation, typically an arithmetic or logical calculation using the Arithmetic Logic Unit (ALU).
4. **Memory Access (MEM):** Reads from or writes to data memory. This stage is active only for load and store instructions.
5. **Write-Back (WB):** Writes the result of the operation (from the EX or MEM stage) back into the destination register in the register file.

While pipelining increases performance, it also introduces challenges known as hazards (data, control, and structural), which can lead to incorrect execution if not properly managed. The following sections will detail the specific implementation of these stages and the logic designed to handle these hazards.

2 Design and Implementation

This section describes the detailed hardware design of the 5-stage pipelined RISC-V processor. The entire CPU is implemented within a single SystemVerilog file, 'riscvpipeline.sv'. This file contains a top-level module that encapsulates the pipeline logic and instantiates all the necessary functional sub-modules, such as the ALU, memories, and control units.

2.1 Top-Level Architecture

The overall structure of the processor follows the classic 5-stage RISC pipeline datapath, as shown in the block diagram in Figure 1. The design is divided into five main stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write-Back (WB).

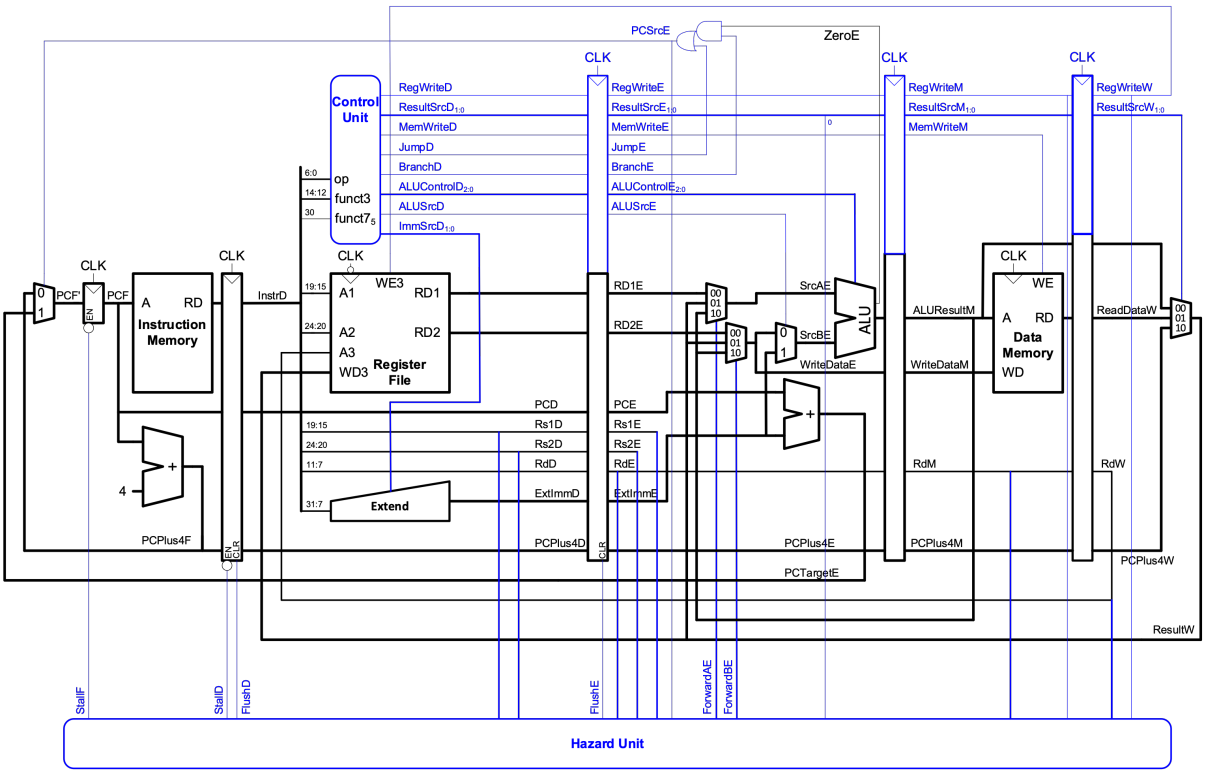

Figure 1: Block Diagram of the 5-Stage Pipelined RISC-V CPU.

The processor's architecture is based on the general datapath shown in Figure 1, which was used as a reference model. While this diagram accurately represents the overall structure, our final design incorporates several enhancements to correctly implement the target instruction set.

The main differences are in the control flow logic. The ALU in our design outputs not only a `zero` flag but also `sign`, `carry`, and `overflow` flags. These are used by a more sophisticated control logic to compute the `PCSrc` signal, which determines the outcome of all conditional branches. Consequently, the simple logic gate shown in the diagram is replaced by a more complex conditional block, which will be detailed later. Furthermore, the PC selection multiplexer in the IF stage is a 3-input MUX, not a 2-input one, as it must also handle the ALUResult as a source for the `jalr` instruction's target address.

Data and control signals flow from one stage to the next through a series of pipeline registers (labeled in the diagram as the large vertical blocks between stages). These registers are updated on each rising edge of the clock and are crucial for isolating the stages and allowing for parallel instruction processing.

The main components visible in the diagram are:

- **Instruction and Data Memory**: Separate memories for instructions (‘imem‘) and data (‘dmem‘), typical of a Harvard architecture.
- **Register File**: Contains the 32 general-purpose registers. It has two read ports (for ‘rs1‘ and ‘rs2‘) and one write port (for ‘rd‘).
- **Control Unit**: Decodes the instruction’s opcode in the ID stage to generate the control signals (e.g., `RegWrite`, `MemWrite`, `ALUSrc`) that will be used in subsequent stages. In this implementation, it is split into a ‘maindec‘ and an ‘aludec‘ module.
- **Hazard Unit**: Implements the logic for forwarding, stalling, and flushing the pipeline to resolve data and control hazards. It generates key signals like `ForwardAE`, `ForwardBE`, `StallD`, and `FlushE`.
- **ALU (Arithmetic Logic Unit)**: Located in the EX stage, it performs all arithmetic and logical operations.

In the SystemVerilog implementation, the pipeline registers are defined as packed structs, which cleanly group all the signals that need to be passed between stages. For example, the register between the ID and EX stages is defined as shown in Listing 1.

```
typedef struct packed {
    logic        RegWrite;
    logic [1:0]  ResultSrc;
    logic        MemWrite;
    logic        Jump;
    logic        Branch;
    logic [3:0]  ALUControl;
    logic        ALUSrc;
    logic [31:0] Rs1;
    logic [31:0] Rs2;
    logic [31:0] RD1;
    logic [31:0] RD2;
    logic [31:0] PC;
    logic [4:0]  Rd;
    logic [31:0] ImmExt;
    logic [31:0] PCPlus4;
    logic [6:0]  op;
    logic [2:0]  funct3;
} ID_EX_reg;

ID_EX_reg ID_EX, ID_EX_next;
```

Listing 1: ID/EX pipeline register definition.

The top-level module, ‘riscvpipeline‘, connects all these components. Listing 2 shows how the main functional units are instantiated and wired together within the design. The pipeline logic itself is implemented using combinational and sequential blocks directly inside the ‘riscvpipeline‘ module, organized by stage.

```
// Memory instances
imem imem_unit (PCCurrent, InstrImem);
dmem dmem_unit (clk, dmem_we, dmem_addr, dmem_wd, dmem_byte_enabler,
    dmem_rd);

// Register File instance
regfile regfile_unit (clk, rf_we, rf_read_addr1, rf_read_addr2,
                      rf_write_addr, rf_write_data, rf_read_data1,
    rf_read_data2);

// Control logic instances
maindec maindec_unit (op, resultsrc, memwrite, ...);
aludec aludec_unit (op, funct3, funct7b5, aluop, alucontrol);

// Immediate generation instance
extend extend_unit (imm_src, immsrc, imm_extended);

// ALU instance
alu alu_unit (alu_src_a, alu_src_b, alu_alucontrol, ...);

// Hazard detection instance
hazard_unit hazard (hazard_Rs1D, hazard_Rs2D, ...);
```

Listing 2: Instantiation of main functional units.

Pipeline Register Update Logic. While most of the processor's logic is combinational (calculating results within a stage), the actual progression of instructions through the pipeline is controlled by a single, central sequential block. This 'always_ff' process, shown in Listing 3, acts as the heartbeat of the CPU, updating the state of all pipeline registers on the rising edge of the clock.

```
// Pipeline registers update logic
always_ff @(posedge clk or posedge reset) begin
    if (reset) begin
        IF_ID <= '0;
        ID_EX <= '0;
        EX_MEM <= '0;
        MEM_WB <= '0;
    end
    else if (StallD) begin
        IF_ID <= IF_ID;      // Hold IF/ID register on stall
        // Later stages continue to advance
        ID_EX <= ID_EX_next;
        EX_MEM <= EX_MEM_next;
        MEM_WB <= MEM_WB_next;
    end
    else begin // Normal operation
        IF_ID <= IF_ID_next;
        ID_EX <= ID_EX_next;
        EX_MEM <= EX_MEM_next;
        MEM_WB <= MEM_WB_next;
    end

    // Asynchronous-like flush logic
    if (FlushD) begin
        IF_ID <= '0; // Nullify instruction in ID stage
    end
```

```
    if (FlushE) begin
        ID_EX <= '0; // Nullify instruction in EX stage
    end
end
```

Listing 3: The sequential logic for pipeline register updates.

This logic block correctly manages the pipeline's state under all conditions:

- **Reset:** On reset, all pipeline registers are cleared to zero, ensuring a known starting state.
- **Normal Operation:** In the default case, each pipeline register is loaded with the values computed in the previous stage (`..._next`), moving every instruction one step forward.
- **Stall:** When the `StallD` signal is asserted by the Hazard Unit, a critical modification occurs. The `IF_ID` register is held, preventing the instruction in the ID stage from advancing. Meanwhile, the later stages of the pipeline continue to execute normally. This effectively inserts a one-cycle "bubble" or delay between the ID and EX stages.
- **Flush:** The `FlushD` and `FlushE` signals act as targeted resets. They are asserted after a taken branch or jump to nullify the incorrectly fetched instructions by clearing the respective pipeline registers. This action overwrites any other update for that register in the same clock cycle, ensuring the flushed instruction is replaced by a 'nop'.

This clear separation between the combinational "calculation" logic and the sequential "state update" logic is fundamental to the design of a stable and predictable pipelined processor.

2.2 Pipeline Stages Implementation

The overall functionality of the CPU is distributed across the five pipeline stages. Each stage is implemented as a set of combinational and sequential logic within the main 'riscvpipeline' module. This section details the specific implementation of each stage.

2.2.1 Instruction Fetch (IF) Stage

The Instruction Fetch (IF) stage is the starting point of the pipeline. Its primary responsibility is to fetch the instruction from the instruction memory ('imem') at the address indicated by the Program Counter (PC).

The core of this stage is the logic that determines the address of the next instruction. A key design decision was to centralize the calculation of all control flow target addresses in the Execute (EX) stage. This simplifies hazard management, as any taken branch or jump consistently results in a flush of the IF and ID stages. As shown in Listing 4, the PC update logic is implemented as a multiplexer controlled by the 'PCSrcE' signal, which is generated in the EX stage.

```
// The PC is updated sequentially in a clocked process
always_ff @(posedge clk, posedge reset) begin
    if(reset)
```

```
        PCCurrent <= 32'b0;
    else
        PCCurrent <= PCNext;
end

// Combinational logic to determine the next PC value
always_comb begin
    if (StallF) begin
        PCNext = PCCurrent;  // Freeze PC on stall from Hazard Unit
    end
    else begin
        case(PCSrcE)
            2'b00:   PCNext = PCCurrent + 4; // Default: next sequential instruction
            2'b01:   PCNext = PCTargetE;     // Branch taken or JAL
            2'b10:   PCNext = alu_result;    // JALR target, from ALU result
            default: PCNext = PCCurrent + 4;
        endcase
    end
end
```

Listing 4: PC update and stall logic in the IF stage.

The logic handles three main cases for the next PC:

- **PC + 4**: For standard sequential execution.
- **Branch/JAL Target**: The ‘PCTargetE‘ address is calculated in the EX stage by an adder. This same value is used for both conditional branches and the ‘jal‘ instruction.
- **JALR Target**: As per your design choice, the target for a ‘jalr‘ instruction is taken directly from the ‘alu_result‘ computed in the EX stage. This allows the jump logic to be handled uniformly alongside branches.

Furthermore, the IF stage is controlled by the ‘Hazard Unit‘. If a ‘StallF‘ signal is asserted (typically for a load-use hazard), the PC is not updated, effectively pausing the fetch of new instructions for one clock cycle.

Finally, the fetched instruction (‘InstrImem‘) and the current PC are passed to the Decode stage through the ‘IF_ID‘ pipeline register, as shown in Listing 5.

```
assign IF_ID_next.PC = PCCurrent;
assign IF_ID_next.Instr = InstrImem;
assign IF_ID_next.PCPlus4 = PCCurrent + 4;
```

Listing 5: Passing data to the IF/ID pipeline register.

2.2.2 Instruction Decode (ID) Stage

The Instruction Decode (ID) stage receives an instruction from the IF stage and prepares all the necessary data and control signals for execution. Its main tasks are to decode the instruction, read source operands from the Register File, and generate the immediate value.

The control logic is implemented by two dedicated modules, ‘maindec‘ and ‘aludec‘. These modules receive the instruction’s ‘op‘ field and ‘funct3‘/‘funct7‘ fields (passed from

the ‘IF_ID‘ register) and generate all the control signals that will guide the datapath in the subsequent stages. This is shown in Listing 6.

```
// Instruction fields are extracted from the IF/ID register
assign op = IF_ID.Instr[6:0];
assign funct3 = IF_ID.Instr[14:12];
// ... and other fields

// Control signals are generated by decoder modules
maindec maindec_unit (op, resultsrc, memwrite, branch, ...);
aludec aludec_unit (op, funct3, funct7b5, aluop, alucontrol);

// Immediate value is generated by the extend unit
extend extend_unit (IF_ID.Instr, immsrc, imm_extended);
```

Listing 6: Decoding and control signal generation in the ID stage.

Simultaneously, the stage reads the source registers (‘rs1‘ and ‘rs2‘) from the Register File. A critical aspect of the design is handling the internal timing of the Register File, which typically writes data at the end of a clock cycle after reading it at the beginning. This creates a data hazard if an instruction in the ID stage needs a result that is currently being written back by an instruction in the WB stage.

To solve this, a specific forwarding path is implemented directly at the output of the Register File, as highlighted in Listing 7. This logic acts as a multiplexer: if the register being read (‘rf_read_addr1‘ or ‘rf_read_addr2‘) is the same as the destination register of the instruction in the WB stage (‘MEM_WB.Rd‘), the new value (‘ResultW‘) is forwarded directly. Otherwise, the value read from the Register File (‘rf_read_data1‘/‘rf_read_data2‘) is used.

```
// Forward data from WB stage if a dependency is detected
assign ReadData1_localID = (MEM_WB.Rd == rf_read_addr1 & MEM_WB.RegWrite
                           & rf_read_addr1 != 0) ? ResultW : rf_read_data1;

assign ReadData2_localID = (MEM_WB.Rd == rf_read_addr2 & MEM_WB.RegWrite
                           & rf_read_addr2 != 0) ? ResultW : rf_read_data2;
```

Listing 7: Forwarding logic for the WB-to-ID data hazard.

Finally, all the control signals, register data, and the sign-extended immediate are passed to the Execute stage through the ‘ID_EX‘ pipeline register.

2.2.3 Execute (EX) Stage

The Execute (EX) stage is the computational core of the processor. It is responsible for performing calculations, determining the outcome of branches, and calculating jump target addresses. This stage receives data and control signals from the ID/EX register and produces a result that is passed to the MEM stage.

Operand Selection and Forwarding. The first task of the EX stage is to select the correct operands for the Arithmetic Logic Unit (ALU). A critical feature of this design is the forwarding logic, which mitigates data hazards by providing results from later stages before they are written back to the register file. As shown in Listing 8, two multiplexers

controlled by the `ForwardAE` and `ForwardBE` signals (generated by the `Hazard Unit`) select the appropriate source for operands A and B. The source can be the original register value from the ID stage, the ALU result from the MEM stage (`EX_MEM.ALUResult`), or the final result from the WB stage (`ResultW`).

A second multiplexer is used for operand B to handle immediate values. Controlled by the `ALUSrc` signal from the control unit, it selects either the (potentially forwarded) register value or the sign-extended immediate (`ID_EX.ImmExt`).

```
// Forwarding MUX for operand A (rs1)
assign ForwardedA = ForwardAE[1] ? ResultW :         // Forward from WB
                    ForwardAE[0] ? EX_MEM.ALUResult:// Forward from MEM
                    ID_EX.RD1;                      // Use value from ID

// Forwarding MUX for operand B (rs2)
assign ForwardedB = ForwardBE[1] ? ResultW :
                    ForwardBE[0] ? EX_MEM.ALUResult :
                    ID_EX.RD2;

// MUX to select between register value and immediate for ALU source B
assign SrcB = ID_EX.ALUSrc ? ID_EX.ImmExt : ForwardedB;

// Final ALU inputs
assign alu_src_a = ForwardedA;
assign alu_src_b = SrcB;
```

Listing 8: Forwarding logic and ALU operand selection.

Control Flow Logic. A key responsibility of the EX stage is to resolve all control flow instructions. This design centralizes this logic into the generation of a single 2-bit signal, `PCSrcE`, which instructs the IF stage on which PC to fetch next. Listing 9 shows the complex conditional logic that evaluates the instruction type and ALU flags (zero, sign, carry and overflow) to make this decision.

```
// Logic to determine the next PC source based on instruction and ALU flags
assign PCSrcE = (ID_EX.op == 7'b1100111) ? 2'b10 : // jalr
                (ID_EX.Branch)           ? {1'b0, ( // branches
                    ((ID_EX.funct3 == 3'b000) && alu_zero) |  //beq
                    ((ID_EX.funct3 == 3'b001) && !alu_zero) |  //bne
                    ((ID_EX.funct3 == 3'b100) && (alu_sign ^ alu_overflow)) |   //blt
                    ((ID_EX.funct3 == 3'b101) && !(alu_sign ^ alu_overflow)) |  //bge
                    ((ID_EX.funct3 == 3'b110) && !alu_carry) |  //bltu
                    ((ID_EX.funct3 == 3'b111) && alu_carry)  //bgeu
                )} :
                (ID_EX.Jump)             ? 2'b01 :  // jal
                                           2'b00;  // default
```

Listing 9: Generation of the PC selection signal (PCSrcE).

If a branch condition is met (e.g., ‘alu_zero‘ is true for a ‘beq‘) or if the instruction is an unconditional jump (‘jal‘), `PCSrcE` signals the IF stage to take the calculated target address. For ‘jalr‘, a different code is used to select the ALU result as the target. If none of these conditions are met, `PCSrcE` remains at its default value, indicating sequential execution (‘PC + 4‘). This centralized approach ensures that all control flow changes are managed from a single point, triggering the necessary flushes in the ‘Hazard Unit‘.

Generalized ALU Result Path. One of the most important design decisions in this processor is how the final result of an instruction is passed through the pipeline. Instead of creating separate data paths for ALU results, memory data, and jump/link addresses, this design cleverly reuses the `ALUResult` field of the pipeline registers for multiple purposes.

As shown in Listing 10, the signal passed to the `EX_MEM_next.ALUResult` is not always the direct output of the ALU. For instructions like ‘lui‘, ‘auipc‘, ‘jal‘, and ‘jalr‘, this field is used to carry their specific results (the immediate value or the calculated target/return address).

```
assign ALUResult_localEX = (ID_EX.op == 7'b0110111) ? ImmExt_localEX : // lui
                           (ID_EX.op == 7'b0010111) ? PCTargetE : // auipc
                           (ID_EX.op == 7'b1101111) ? PCTargetE : // jal
                           (ID_EX.op == 7'b1100111) ? PCJalr : // jalr
                           alu_result; // Default case
```

Listing 10: Selection of the value for the ALUResult path.

This architectural choice offers two significant advantages:

1. **Simplified Forwarding Logic**: In subsequent clock cycles, the forwarding unit only needs to monitor one signal path (‘EX_MEM.ALUResult‘ and ‘MEM_WB.ALU-Result‘) to get the result of any register-writing instruction, regardless of its type. This avoids the need for a more complex forwarding mux that would have to select between the ALU output, PC-related values, and memory data.

2. **Simplified Write-Back Stage**: The multiplexer in the WB stage is also simplified. It only needs to choose between the value from the "ALU Result" path, the value from the "Memory Data" path and the value from ‘PC+4‘ path, instead of handling additional inputs from ‘ImmExtend‘ or ‘PCTarget‘ that are now routed through the ‘ALUResult‘ path.

Finally, the EX stage also prepares the data to be written for store instructions (`sw`, `sh`, `sb`) and passes all relevant results and control signals to the MEM stage via the `EX_MEM` pipeline register.

2.2.4 Memory Access (MEM) Stage

The Memory Access (MEM) stage is responsible for all interactions with the data memory (‘dmem‘). It performs load operations (reading from memory) and store operations (writing to memory). For all other instructions, this stage simply acts as a passthrough, forwarding the result from the EX stage to the WB stage.

Store Logic and Byte Enabling. For store instructions (‘sw‘, ‘sh‘, ‘sb‘), this stage writes data into the ‘dmem‘. A key challenge is that the RISC-V ISA supports writing individual bytes and half-words, but the data memory is typically organized into 32-bit words. To handle this, a technique called *byte enabling* is used. A 4-bit signal, `ByteEnabler`, is generated to control which of the four bytes within a 32-bit memory word should be modified.

As shown in Listing 11, the logic calculates `ByteEnabler` based on the instruction type (‘funct3‘) and the two least significant bits of the memory address (‘EX_MEM.ALUResult [1:0]‘), which represent the exact byte address of the ‘dmem‘. For a ‘sw‘, all four bytes are enabled. For ‘sh‘ and ‘sb‘, only the specific one or two bytes at the target address are enabled for writing.

```
// Byte Enabler logic for partial writes to memory
assign ByteEnabler = (EX_MEM.MemWrite) ?
  ((EX_MEM.funct3 == 3'b010) ? 4'b1111 : // sw: enable all 4 bytes
  (EX_MEM.funct3 == 3'b001) ? (4'b0011 << (EX_MEM.ALUResult[1] * 2)) : // sh
  (EX_MEM.funct3 == 3'b000) ? (4'b0001 << EX_MEM.ALUResult[1:0]) : // sb
  4'b0000) : 4'bxxxx;

// Connecting signals to the data memory module
assign dmem_we = EX_MEM.MemWrite;
assign dmem_addr = EX_MEM.ALUResult;
assign dmem_wd = EX_MEM.WriteData; // Data prepared in EX stage
assign dmem_byte_enabler = ByteEnabler;
```

Listing 11: Byte enable logic for store instructions.

Load Logic. For load instructions (‘lw‘, ‘lh‘, ‘lbu‘, etc.), this stage reads a 32-bit word from ‘dmem‘ and then extracts the correct byte or half-word. The logic, shown in Listing 12, first retrieves the relevant portion from the 32-bit data read (‘dmem_rd‘) based on the address. Then, it correctly expands this portion to a 32-bit value. For signed loads (‘lb‘, ‘lh‘), the most significant bit of the extracted data is used for sign extension. For unsigned loads (‘lbu‘, ‘lhu‘), the value is zero-extended. This final 32-bit value, `LoadData`, is what will be written to the register file.

```
// Extract the correct byte from the 32-bit word read from memory
assign ByteFromMem = (EX_MEM.ALUResult[1:0] == 2'b00) ? dmem_rd[7:0] :
                     (EX_MEM.ALUResult[1:0] == 2'b01) ? dmem_rd[15:8] :
                     // ... other cases

// Assemble the final 32-bit data to be written back
assign LoadData = (EX_MEM.funct3 == 3'b010) ? dmem_rd : // lw
                  (EX_MEM.funct3 == 3'b000) ? {{24{ByteFromMem[7]}}, ByteFromMem} : // lb (sign-extended)
                  (EX_MEM.funct3 == 3'b100) ? {{24{1'b0}}, ByteFromMem} : // lbu (zero-extended)
                  // ... logic for lh and lhu ...
                  dmem_rd;
```

Listing 12: Data extraction and extension for load instructions.

All necessary data, including the `ALUResult` and the newly loaded data, are then passed to the final stage through the `MEM_WB` pipeline register.

2.2.5 Write-Back (WB) Stage

The Write-Back (WB) stage is the final step in the pipeline. Its sole purpose is to write the result of an instruction back into the Register File. This stage is relatively simple, as most of the complex calculations have been handled in previous stages.

The core of the WB stage is a multiplexer controlled by the `ResultSrc` signal, which was originally generated in the ID stage and propagated through the pipeline. As shown in Listing 13, this MUX selects the correct data to be written to the destination register.

```
assign ResultW = (MEM_WB.ResultSrc == 3'b00) ? MEM_WB.ALUResult :
                 (MEM_WB.ResultSrc == 3'b01) ? MEM_WB.ReadData :
                 (MEM_WB.ResultSrc == 3'b10) ? MEM_WB.PCPlus4 :
                 32'bx;
```

Listing 13: Result selection logic in the WB stage.

The possible sources for the write-back data are:

- The result from the ALU path (`MEM_WB.ALUResult`), which, due to the design choices in the EX stage, carries the result for all arithmetic, logical, and jump-and-link instructions.
- The data read from memory (`MEM_WB.ReadData`) for load instructions.
- The value of PC+4 (`MEM_WB.PCPlus4`), used for jump-and-link instructions to store the return address.

The final selected value, `ResultW`, along with the destination register address `MEM_WB.Rd` and the write enable signal `MEM_WB.RegWrite`, are fed to the Register File, completing the instruction's execution.

2.3 Key Functional Modules

Beyond the pipeline structure itself, the processor's functionality relies on several key hardware modules. This section provides a brief overview of the implementation of the most important ones: the control logic, the data memory, the ALU, and the hazard unit.

2.3.1 Control Logic (`maindec` and `aludec`)

The control signals that orchestrate the datapath are generated by a combination of two decoder modules: 'maindec' and 'aludec'. This separation of concerns simplifies the decoding logic.

The `maindec` module is the primary decoder. It takes the 7-bit instruction opcode as input and produces the main control signals that are independent of the instruction's 'funct3' or 'funct7' fields. As shown in Listing 14, it uses a single 'case' statement to output a bit-vector containing signals like `RegWrite`, `MemWrite`, `Branch`, and `ALUSrc`. It also generates a 2-bit `ALUOp` signal that serves as a preliminary instruction to the 'aludec' module.

```
module maindec(input  logic [6:0] op, ...);
  logic [11:0] controls;
  assign {RegWrite, ImmSrc, ALUSrc, ...} = controls;

  always_comb
```

```
    case(op)
      // RegWrite_ImmSrc_ALUSrc_MemWrite_ResultSrc_Branch_ALUOp_Jump
      7'b0000011: controls = 12'b1_000_1_0_01_0_00_0; // lw
      7'b0100011: controls = 12'b0_001_1_1_00_0_00_0; // sw
      7'b0110011: controls = 12'b1_xxx_0_0_00_0_10_0; // R-type
      // ... other instructions
      default:    controls = 12'bx_xx_x_x_xx_x_xx_x;
    endcase
endmodule
```

Listing 14: Main decoder logic based on the instruction opcode.

The `aludec` module, or ALU decoder, is responsible for generating the final 4-bit `ALUControl` signal that directly commands the ALU. It uses the ‘ALUOp‘ signal from ‘maindec‘ along with the ‘funct3‘ and ‘funct7‘ fields to determine the specific operation (e.g., distinguishing between ‘add‘ and ‘sub‘ for R-type instructions). This two-level decoding approach is a classic and efficient way to manage control logic in a RISC processor.

2.3.2 Data Memory (`dmem`)

The `dmem` module models a 32-bit wide, word-addressable data memory. While reads are straightforward, writing data requires careful handling of sub-word accesses as specified by the RISC-V ISA for ‘sb‘ (store byte) and ‘sh‘ (store half-word) instructions.

As previously introduced, this is achieved using a 4-bit write enable signal, `bwe` (byte write enable). The internal logic of the ‘dmem‘ module, shown in Listing 15, uses this signal within a clocked process. On the rising edge of the clock, if the main write enable signal (‘we‘) is active, the module checks each bit of the ‘bwe‘ signal. A specific byte within the target 32-bit memory word is updated only if its corresponding enable bit is high. This ensures that ‘sb‘ and ‘sh‘ instructions modify only the intended bytes, leaving the rest of the word untouched.

```
module dmem(input logic clk, we, input logic [3:0] bwe, ...);
  logic [31:0] RAM[63:0];

  always_ff @(posedge clk)
    if (we) begin
      if (bwe[0]) RAM[a[31:2]][7:0]   <= wd[7:0];   // Update byte 0
      if (bwe[1]) RAM[a[31:2]][15:8]  <= wd[15:8];  // Update byte 1
      if (bwe[2]) RAM[a[31:2]][23:16] <= wd[23:16]; // Update byte 2
      if (bwe[3]) RAM[a[31:2]][31:24] <= wd[31:24]; // Update byte 3
    end

  assign rd = RAM[a[31:2]]; // Word-aligned read
endmodule
```

Listing 15: Internal byte-enabled write logic of the dmem module.

2.3.3 Arithmetic Logic Unit (`alu`)

The Arithmetic Logic Unit (`alu`) is the combinational circuit that performs all arithmetic and logical operations required by the instruction set. It takes two 32-bit operands (‘a‘ and ‘b‘) and a 4-bit ‘alucontrol‘ signal as input, producing a 32-bit result and several status flags.

The core of the ALU is a ‘case‘ statement that selects the operation based on the ‘alucontrol‘ input, as shown in Listing 16. The design implements operations such as

addition, subtraction, bitwise AND, OR, XOR, and various shift and comparison instructions. For addition and subtraction, a single adder/subtractor circuit is used, where subtraction is implemented as addition of the two's complement of the second operand.

```
module alu(input [31:0] a, b, input [3:0] alucontrol, ...);
  // ... signal declarations ...

  always_comb
    case (alucontrol)
      4'b0000: result = a + b;            // add, addi
      4'b0001: result = a - b;            // sub (sum with 2's complement)
      4'b0010: result = a & b;            // and, andi
      4'b0101: result = ($signed(a) < $signed(b)); // slt, slti
      4'b0110: result = a << b[4:0];      // sll, slli
      // ... other operations ...
      default: result = 32'bx;
    endcase

  // Status flags calculation
  assign zero = (result == 32'b0);
  assign sign = result[31];
  // ... overflow and carry logic ...
endmodule
```

Listing 16: Main operational logic of the ALU.

In addition to the main result, the ALU also computes four status flags: 'zero' (true if the result is zero), 'sign' (the most significant bit of the result), 'overflow', and 'carry'. These flags are crucial for evaluating conditional branches in the EX stage.

2.3.4 Hazard Unit (`hazard_unit`)

The `hazard_unit` is arguably the most critical component for ensuring the correctness of the pipelined execution. It is a purely combinational module that detects all potential data and control hazards and generates the necessary control signals—stalls, flushes, and forwarding directives—to resolve them. The entire logic is contained within a single 'always_comb' block, detailed in Listing 17.

```
always_comb begin
  // Default values
  ForwardAE = 2'b00;
  ForwardBE = 2'b00;

  // --- Forwarding Logic for Rs1E ---
  // Hazard: EX stage needs a result from MEM stage
  if ((Rs1E == RdM) & RegWriteM & (Rs1E != 0))
    ForwardAE = 2'b01;
  // Hazard: EX stage needs a result from WB stage
  else if ((Rs1E == RdW) & RegWriteW & (Rs1E != 0))
    ForwardAE = 2'b10;

  // --- Forwarding Logic for Rs2E ---
  if ((Rs2E == RdM) & RegWriteM & (Rs2E != 0))
    ForwardBE = 2'b01;
  else if ((Rs2E == RdW) & RegWriteW & (Rs2E != 0))
    ForwardBE = 2'b10;

```

```
  // --- Stall and Flush Logic ---
  // Stall for load-use hazard: ID stage is using a register that
  // EX stage is loading into from memory.
  StallF = ((Rs1D == RdE) | (Rs2D == RdE)) & (ResultSrcE == 3'b01); //
   lw hazard
  StallD = StallF;

  // Flush EX stage if a stall occurs or a branch is taken
  FlushE = StallF | (PcSrc != 2'b00);
  // Flush ID stage if a branch is taken
  FlushD = (PcSrc != 2'b00);
end
```

Listing 17: Complete logic of the Hazard Unit.

The logic can be broken down into three primary functions:

Data Forwarding. The unit first checks for data hazards between the instruction in the EX stage and the instructions in the MEM and WB stages.

- It compares the source registers in EX (`Rs1E`, `Rs2E`) with the destination register in MEM (`RdM`). If there is a match and the instruction in MEM is writing to a register (`RegWriteM`), it sets the forwarding signals (`ForwardAE` or `ForwardBE`) to '2'b01'. This directs the MUX in the EX stage to take the result from the MEM stage.
- If no MEM-stage hazard is found, it then checks for a dependency with the WB stage (`RdW`). This case has lower priority because the MEM stage data is more recent. If a WB-stage dependency is found, the forwarding signals are set to '2'b10'.
- The condition 'Rs1E != 0' ensures that no forwarding occurs for reads from register 'x0', which is always zero.

Load-Use Stall. A special case of data hazard, the load-use hazard, cannot be resolved by forwarding alone. If an instruction in the ID stage depends on the result of a 'lw' instruction currently in the EX stage, the data will only be available from memory at the end of the MEM stage. The pipeline must be stalled for one cycle. The unit detects this by checking if a source register in ID (`Rs1D` or `Rs2D`) matches the destination of the instruction in EX (`RdE`), and if that instruction is a load (`ResultSrcE == 2'b01`). If so, it asserts `StallF` and `StallD`, which freezes the PC and the IF/ID register.

Control Hazard Flush. Whenever a control flow instruction (a taken branch or a jump) is processed in the EX stage, the instructions that were fetched based on the old, sequential PC are incorrect and must be discarded. The `hazard_unit` detects this condition when `PcSrc != 2'b00`.

- It asserts `FlushD`, which clears the IF/ID pipeline register, effectively converting the fetched instruction into a 'nop'.
- It also asserts `FlushE`, which clears the ID/EX register. This is necessary both for control hazards and to prevent the stalled instruction (in a load-use case) from progressing incorrectly.

This comprehensive logic ensures that the pipeline remains synchronized and produces correct results despite the parallel execution of instructions.

3 Verification and Results

After the design and implementation phase, the processor's functionality was rigorously verified to ensure it behaves as expected. This section outlines the methodology used for testing and presents a series of test cases that demonstrate the correct operation of the CPU, including its hazard management logic.

3.1 Test Methodology

The verification strategy involved a custom toolchain to generate machine code, a SystemVerilog testbench to simulate the CPU, and waveform analysis to inspect the results.

Assembly and Machine Code Generation. A comprehensive test program was written in RISC-V assembly language (‘instructions.txt‘). This program was designed to test every implemented instruction and to trigger specific data and control hazard scenarios. To convert the human-readable assembly code into machine code understandable by the CPU, a simple assembler script was written in Python (`assembler.py`). This script parses the assembly file and outputs a text file, `machinecode.txt`, containing the 32-bit instruction codes in hexadecimal format. This file is then loaded by the instruction memory module (‘imem‘) at the start of the simulation.

Simulation Environment. A SystemVerilog testbench (‘testbench.sv‘) was created to simulate the CPU's operation. The testbench's responsibilities are minimal but crucial:

- It instantiates the top-level ‘riscvpipeline‘ module (the Device Under Test, or DUT).
- It provides a clock signal with a fixed period.
- It asserts the ‘reset‘ signal for the first few cycles to initialize the pipeline to a known state.
- It uses SystemVerilog's built-in ‘$dumpfile‘ and ‘$dumpvars‘ commands to generate a Value Change Dump (‘.vcd‘) file, which logs the state of all internal signals of the CPU throughout the simulation.

Waveform Analysis. The primary method for verification was the manual analysis of the waveforms produced by the simulation, using the GTKWave tool. This approach allows for a deep inspection of the processor's internal state on a cycle-by-cycle basis. The general verification process for any given instruction involves:

1. Observing the `PCCurrent` and `InstrImem` signals to track the flow of the program and see which instruction is being fetched and decoded.
2. For arithmetic, logical, or load instructions, checking the value of `ResultW` four clock cycles after the instruction is fetched to confirm that the calculated result is correct and is being written to the right destination register (`MEM_WB.Rd`).
3. For store instructions, monitoring the data memory's internal signals, such as `dmem_we` (write enable) and `dmem_wd` (write data), to ensure the correct value is being written to the correct address.

4. For hazard scenarios, observing the output signals of the `hazard_unit`, such as `ForwardAE`, `StallD`, and `FlushE`, to confirm that hazards are being correctly detected and resolved.

The manual verification process was made significantly more efficient and reliable by a detailed documentation file, `detailed_instructions.txt`. As shown in the provided code, this file lists every instruction in the test program, annotated with:

- Its memory address and machine code equivalent.
- A human-readable description of the operation being performed.
- The expected result or state change (e.g., `t0 = 0x12345678` or `Branch -> Taken`).

This document served as a direct "answer key" during waveform analysis, allowing for a quick and accurate comparison between the expected behavior and the actual signal values observed in GTKWave.

Given the length of the test program, this report will not show the verification of every single instruction. Instead, the following subsection presents four representative test cases that demonstrate the correct functionality of the processor's core features.

3.2 Test Cases and Analysis

This section presents four key test cases from the verification process. Each case is chosen to demonstrate a specific, critical feature of the CPU's pipeline and hazard management system.

3.2.1 Case 1: Data Forwarding from MEM to EX

This test demonstrates the functionality of the forwarding unit in resolving a standard data hazard without stalling the pipeline. The sequence under test is:

```
    addi t2, x0, -1
    addi t3, t2, 16
```

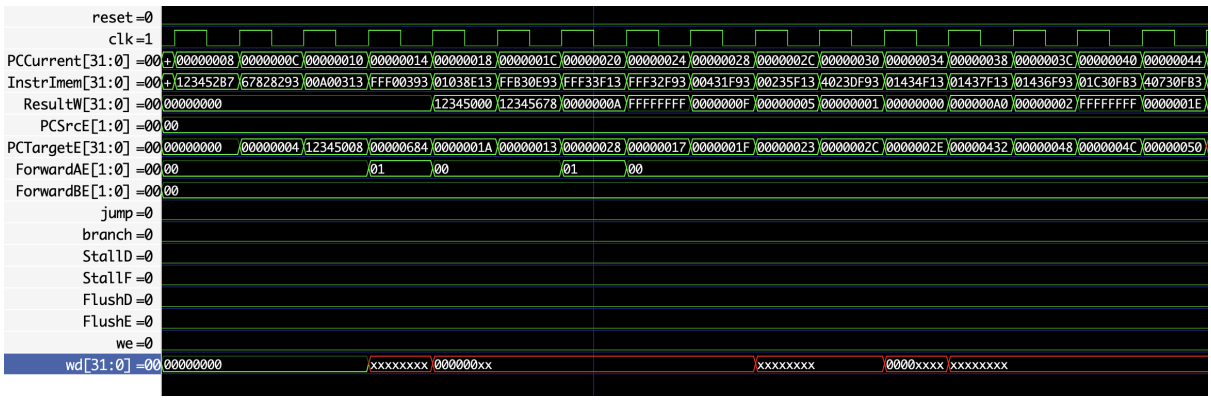

Figure 2: Waveform showing a data hazard resolved by forwarding.

The analysis of the waveform in Figure 2 shows:

1. At PC `0x014`, the first instruction, `addi t2, x0, -1` (machine code `0xfff00393`), is fetched.

2. Three clock cycles later, this instruction is in the MEM stage. Its result, ‘-1‘ (`0xFFFFFFFF`), is available at the output of the ALU, but has not yet been written back to the register file.

3. In this same cycle, the second instruction, `addi t3, t2, 16`, is in the EX stage. It needs the value of register `t2`.

4. The `hazard_unit` correctly detects this dependency. We can see the signal `ForwardAE` becomes `2'b01`, indicating that the result from the MEM stage should be forwarded directly to the first input of the ALU in the EX stage.

5. As a result, the ‘addi‘ instruction calculates ‘-1 + 16‘ correctly. Four cycles after the instruction `addi t3, t2, 16` is fetched (PC `0x018`), its result, ‘15‘ (`0x0F`), appears on the `ResultW` bus to be written back.

This test case successfully verifies that the forwarding logic correctly handles EX-to-EX data dependencies, preventing a stall and maintaining pipeline throughput.

3.2.2 Case 2: Load-Use Hazard and Pipeline Stall

This is a critical test case that demonstrates the processor's ability to handle a load-use hazard, which requires a combination of stalling and forwarding. The sequence is:

```
    lw t1, 8(sp)          // Load into t1
    addi t0, t1, -6       // Immediately use t1
```

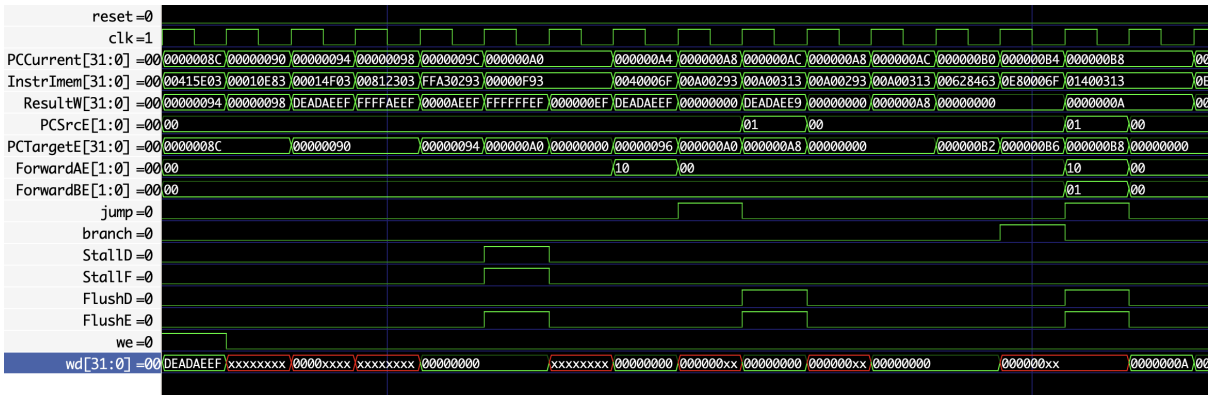

Figure 3: Waveform showing a load-use hazard causing a stall and a flush.

The waveform in Figure 3 is analyzed as follows:

1. At PC `0x...98`, the `lw` instruction is fetched.

2. At the next cycle (PC `0x...9C`), the dependent `addi` instruction is fetched.

3. One cycle later, the ‘lw‘ is in the EX stage and the ‘addi‘ is in the ID stage. The `hazard_unit` detects that the instruction in ID needs a result from a ‘lw‘ instruction in EX. This data will not be ready until the end of the MEM stage.

4. The `hazard_unit` asserts `StallD` and `StallF` for one cycle. This freezes the PC and the IF/ID register, preventing new instructions from being fetched and the ‘addi‘ from advancing.

5. Simultaneously, `FlushE` is asserted to inject a "bubble" (a 'nop') into the EX stage, neutralizing the instruction that was in the ID stage during the stall.

6. In the following cycle, the stall is released. The 'lw' instruction has moved to the MEM stage, and its data is now available to be forwarded. The 'addi' instruction re-enters the EX stage and can now receive the correct value of 't1' via forwarding.

7. As a result, the 'addi' instruction calculates '0xDEADAEEF - 6' correctly. Four cycles after the instruction `addi t0, t1, -6` is fetched (PC `0x09C`), its result, `0xDEADAEE9`, appears on the `ResultW` bus to be written back.

This test confirms that the load-use hazard detection logic is working correctly, preserving program integrity by enforcing a one-cycle stall.

3.2.3 Case 3: Control Hazard (Branch Taken)

This test verifies the correct handling of control hazards. A `bne` instruction is executed and the branch is taken, which must cause the pipeline to flush the incorrectly fetched subsequent instruction.

```
    addi t0, x0, 10
    addi t1, x0, 20
    bne t0, t1, bne_taken_ok // Branch is taken
    jal x0, fail_handler     // This instruction must be flushed
    ...
bne_taken_ok:
    addi t1, x0, 10
```

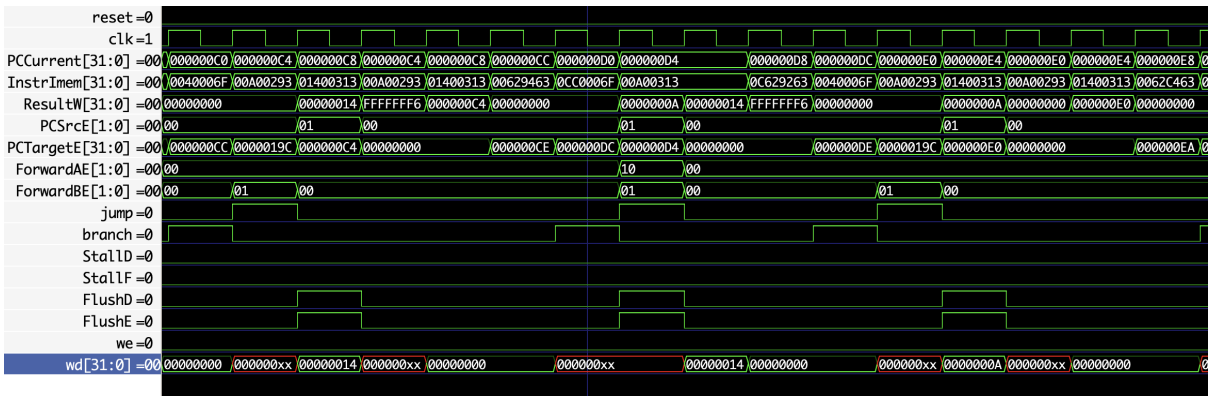

Figure 4: Waveform showing a taken branch and subsequent pipeline flush.

The sequence of events in Figure 4 is as follows:

1. At PC `0x...0CC`, the `bne` instruction is fetched.

2. While 'bne' is in the ID stage, the processor speculatively fetches the next sequential instruction, `jal x0, fail_handler`, at PC `0x...0D0`.

3. When the `bne` instruction reaches the EX stage, the branch condition is evaluated and found to be true. The control logic sets `PCSrcE` to `2'b01` to signal a jump.

4. The `hazard_unit` detects that `PCSrcE` is not '00' and immediately asserts `FlushD` and `FlushE`.

5. The `FlushD` signal clears the IF/ID register, effectively replacing the speculatively fetched ‘jal‘ instruction with a ‘nop‘.

6. On the next rising clock edge, `PCCurrent` is updated not with ‘PC+4‘, but with the branch target address from `PCTargetE` (`0x...94`). The pipeline resumes fetching from the correct location.

7. From the sequence of `PCCurrent` is possible to see the effectiveness of branch logic.

This demonstrates that the control hazard mechanism is functioning correctly, ensuring that the processor follows the correct execution path after a branch.

3.2.4 Case 4: Final Program Verification

Finally, this test case verifies that the entire test program runs to completion and reaches the intended success state, without ever branching to the `fail_handler`.

```
test_jal:
    jal ra, jal_target
    jal x0, pass_handler

jal_target:
    jal x0, test_jalr

test_jalr:
    jalr t3, 0(ra)

jalr_return_ok:
    jal x0, fail_handler

pass_handler:
    addi t6, x0, 1
pass_loop:
    jal x0, pass_loop

fail_handler:
    addi t6, x0, -1
fail_loop:
    jal x0, fail_loop
```

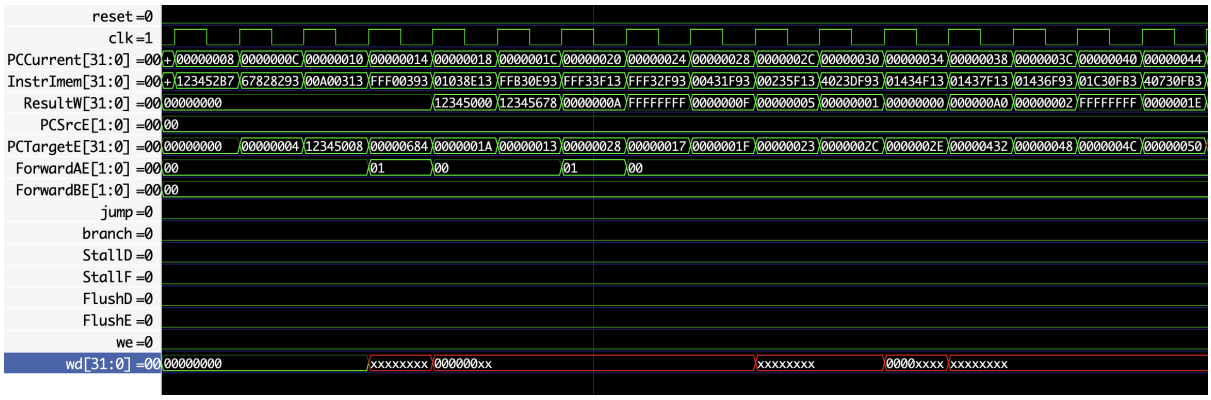

Figure 5: Waveform showing the execution of the final instruction in `pass_handler`.

Figure 5 shows the final moments of the test program's execution. At PC `0x...194`, the instruction `addi t6, x0, 1` (machine code `0x00100f93`) is executed. Four clock cycles later, the `ResultW` bus shows the value `1`, which is written into the destination register for `t6`. The successful execution of this instruction serves as the final confirmation that all preceding tests passed and the CPU is fully functional for the implemented instruction set.

4 Conclusion

4.1 Summary of Work

This project successfully achieved the goal of designing, implementing, and verifying a 32-bit, 5-stage pipelined processor supporting a significant subset of the RISC-V (RV32I) instruction set. The entire CPU was developed in SystemVerilog, following the classic pipeline architecture of Fetch, Decode, Execute, Memory, and Write-Back stages. A key part of the design was the implementation of a comprehensive hazard unit. This unit effectively manages data hazards through forwarding and stalling, and control hazards through pipeline flushing, ensuring correct program execution. The processor's functionality was confirmed through a series of targeted tests, with results analyzed in GTKWave, proving that all implemented instructions and hazard scenarios operate as intended.

4.2 Challenges and Future Work

One of the main challenges during this project was debugging the hazard detection logic, particularly for complex cases like the load-use dependency and the correct flushing after a taken branch. These issues were resolved through careful, cycle-by-cycle waveform analysis, which helped to refine the logic in the hazard unit.

This project serves as a solid foundation for further development. Possible future work could include:

- **Expanding the ISA**: Implementing more instructions from the RISC-V standard, such as those from the "M" extension for integer multiplication and division, to increase the processor's computational capabilities.
- **FPGA Implementation and Validation**: Synthesizing the current design and implementing it on a physical FPGA board. This step would move the project from simulation to a real hardware environment, allowing for real-world performance testing and interaction with physical I/O peripherals like LEDs or a UART for communication.

These potential improvements would transform the current design into a more complete and robust microprocessor system.